PRABIN YADAV
CH.SC.U4CSE25275

Question_10:

Question description

Hassan gets a job in a software company in Hyderabad. The training period for the first three months is 20000 salary. Then incremented to 25000 salaries.

Training is great but they will give you a programming task every day in three months. Hassan must finish it in the allotted time. His teammate Jocelyn gives him a task to complete the concept of Postfix to Prefix Conversion for a given expression. can you help him?

Functional Description:

- Read the Prefix expression in reverse order (from right to left)
- If the symbol is an operand, then push it onto the Stack
- If the symbol is an operator, then pop two operands from the Stack
  Create a string by concatenating the two operands and the operator after them.
  string = operand1 + operand2 + operator
  And push the resultant string back to Stack
- Repeat the above steps until end of Prefix expression.

Constraints

the input should be a expressions

Input Format

Single line represents the postfixed expressions

Output Format

Single line represents the prefix expression

✓ Logical Test Cases

Test Case 1

INPUT (STDIN)

```
ABC/-AK/L-*
```

EXPECTED OUTPUT

```
*-A/BC-/AKL
```

Test Case 2

INPUT (STDIN)

```
AB+CD-*
```

EXPECTED OUTPUT

```
*+AB-CD
```

PRABIN YADAV
CH.SC.U4CSE25275

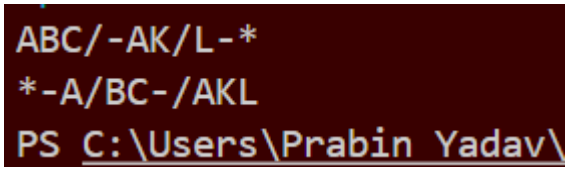
Code:

```cpp
/*
Question_10:
Convert a postfix expression into a prefix expression using a stack.
*/

#include <iostream>
#include <stack>
#include <string>
using namespace std;
string postToPre(string post_exp)
{
    stack<string> s;
    for(int i = 0; i < post_exp.length(); i++)
    {
        char ch = post_exp[i];
        if(isalnum(ch))
        {
            s.push(string(1, ch));
        }
        else
        {
            string op2 = s.top();
            s.pop();
            string op1 = s.top();
            s.pop();
            string result = string(1, ch) + op1 + op2;
            s.push(result);
        }
    }
    return s.top();
}
int main()
{
    string post_exp;
    cin >> post_exp;
    cout << postToPre(post_exp);
    return 0;
}
```

Output:

Test Case_1:



Test Case_2: